

PROJECT REPORT

Visual Product Search Engine

Submitted By

Mayankkumar Satapara - MT2025069

Prashant Sharma - MT2025091

Aditya Dave - MT2025013

M.Tech. (CSE)
IIIT Bangalore

May, 2026

Abstract

This project presents a Visual Product Search Engine that enables users to search for similar fashion products using an input image instead of traditional text-based queries. The system is designed to overcome limitations of keyword search such as inconsistent product descriptions and language mismatch across e-commerce platforms. The proposed pipeline combines YOLO for product localization, BLIP-2 for semantic caption generation, and CLIP for cross-modal image-text embedding generation. The generated embeddings are stored using an HNSW-based vector indexing system to enable efficient and scalable nearest-neighbor retrieval. During online search, the uploaded query image is processed to retrieve visually and semantically similar products from the catalog. The system is trained and evaluated on the DeepFashion In-Shop Clothes Retrieval dataset using standard retrieval metrics including Recall@K, NDCG@K, and mAP@K. Experimental analysis demonstrates that combining visual and semantic features with CLIP fine-tuning significantly improves retrieval accuracy and overall search relevance. :contentReference[oaicite:0]index=0

Contents

Abstract	1
1 Introduction	6
2 System Overview and Architecture	7
2.1 Overall Architecture	7
2.2 Offline Indexing Pipeline	10
2.2.1 Input to the Pipeline	10
2.2.2 YOLO-based Product Detection	10
2.2.3 BLIP-2 Caption Generation	11
2.2.4 CLIP Embedding Generation	12
2.2.5 Embedding Fusion and Normalization	12
2.2.6 HNSW Vector Indexing	13
2.3 Online Query Pipeline	15
2.3.1 Query Image Upload	15
2.3.2 Product Localization using YOLO	15
2.3.3 CLIP Query Embedding Generation	16
2.3.4 Similarity Retrieval	16
2.3.5 Semantic Re-ranking	17
2.3.6 Final Retrieval Results	17
2.3.7 Workflow of Online Query Pipeline	18
3 Model and Component Explanation	19
3.1 YOLO-based Product Detection	19
3.1.1 Object Detection using YOLO	19
3.1.2 Bounding Box Generation	20
3.1.3 Importance of Cropping	20
3.1.4 Background Noise Reduction	20
3.1.5 Before and After Product Cropping	21
3.2 BLIP-2 Caption Generation	23
3.2.1 Image-to-Text Generation	23
3.2.2 Semantic Understanding	23

3.2.3	Caption Consistency	24
3.2.4	Workflow of BLIP-2 Caption Generation	25
3.3	CLIP Embedding Model	26
3.3.1	Image Encoder	26
3.3.2	Text Encoder	27
3.3.3	Shared Embedding Space	28
3.3.4	Cross-modal Learning	28
3.3.5	Similarity Matching Concept	29
3.3.6	Fine-tuning of CLIP	29
3.3.7	Architecture of CLIP Embedding Model	30
3.4	Embedding Fusion	32
3.4.1	Image Embedding	32
3.4.2	Text Embedding	32
3.4.3	Alpha Weighting	33
3.4.4	Normalization of Embeddings	33
3.5	HNSW Vector Search	34
3.5.1	Approximate Nearest Neighbor Search	34
3.5.2	Fast Similarity Retrieval	34
3.5.3	Graph-based Indexing Structure	35
3.5.4	Workflow of HNSW Vector Search	36
3.5.5	HNSW Graph Structure	37
3.6	BLIP-2 Semantic Re-ranking	38
3.6.1	Limitations of Initial Retrieval	38
3.6.2	Semantic Re-ranking Process	38
3.6.3	Image-Text Matching (ITM) Score	39
3.6.4	Workflow of BLIP-2 Re-ranking	41
4	Results and Discussion	42
4.1	Retrieval Performance	42
4.2	Top-K Retrieval Examples	42
4.3	Good Retrieval Cases	42
4.4	Failure Cases	43
5	UI and Deployment	44
5.1	Objective	44
5.2	System Architecture	44
5.3	Frontend Development Using Streamlit	45
5.3.1	Automatic Clothing Region Extraction	46
5.3.2	Feature Extraction and Retrieval	46

5.3.3	Hugging Face Dataset Hosting	47
5.3.4	Asset Download and Caching	47
5.4	Deployment on Streamlit Community Cloud	48
5.4.1	Final Output	49
5.5	User Interface Demonstration	49
5.6	Project Links	52

List of Figures

2.1	Overall Architecture of the Visual Product Search Engine	9
2.2	Workflow of Offline Indexing Pipeline	14
2.3	Workflow of Online Query Pipeline	18
3.1	Before and After YOLO-based Product Cropping	22
3.2	Architecture of CLIP Embedding Model	30
3.3	Workflow of HNSW Vector Search	36
3.4	Illustration of HNSW Graph-based Indexing	37
3.5	Workflow of BLIP-2 Semantic Re-ranking	40
4.1	Top-K Retrieval Results	43
4.2	Examples of Retrieval Failure Cases	43
5.1	User uploading a query image into the Streamlit application	50
5.2	Automatically generated Full Body, Upper Body, and Lower Body regions	50
5.3	Top-5 retrieved similar products displayed by the system	51
5.4	Error message	51

Chapter 1

Introduction

Visual Product Search is an image-based retrieval system that allows users to search for similar fashion products using an input image instead of traditional text queries. In modern e-commerce platforms, users often encounter products through images on social media or websites but face difficulty finding similar items using keyword-based search systems. Traditional text search depends heavily on manually written titles, tags, and product descriptions, which may vary across sellers and platforms, leading to inaccurate or inconsistent search results.

To address this limitation, this project proposes a multimodal Visual Product Search Engine that combines computer vision and deep learning techniques for efficient product retrieval. The system processes a query image and retrieves visually and semantically similar fashion products from a product database. The proposed pipeline integrates multiple state-of-the-art models to improve retrieval accuracy and semantic understanding.

The system uses YOLO for product detection and localization, which helps isolate the primary clothing item from the background. BLIP-2 is used to generate semantic captions that describe product attributes such as color, style, and clothing type. CLIP is used to generate image and text embeddings in a shared feature space, enabling cross-modal similarity matching between images and generated captions. The fused embeddings are stored using an HNSW (Hierarchical Navigable Small World) indexing structure to support efficient approximate nearest neighbor retrieval for large-scale search.

Overall, the proposed system combines visual understanding, semantic representation, and scalable vector search to create an effective image-based fashion retrieval pipeline.

Chapter 2

System Overview and Architecture

2.1 Overall Architecture

The Visual Product Search Engine is designed as a multimodal retrieval system that combines computer vision, semantic understanding, and vector similarity search for efficient fashion product retrieval. The complete system consists of two major stages: the Offline Indexing Pipeline and the Online Query Pipeline. The system processes product images, generates semantic and visual embeddings, stores them in a vector database, and retrieves similar products based on image similarity and semantic relevance.

In the offline indexing stage, all catalog product images from the dataset are processed and converted into searchable vector embeddings before retrieval. First, YOLO is used to detect and crop the main clothing item from each image to reduce background noise and improve feature extraction quality. The cropped image is then passed to the BLIP-2 caption generation model, which produces a semantic description of the product including attributes such as color, style, and clothing category. After caption generation, CLIP generates both image and text embeddings in a shared feature space. These embeddings are fused into a single normalized representation and stored using an HNSW indexing structure for efficient nearest-neighbor retrieval.

In the online query stage, the user uploads a query image through the application interface. Similar to the offline pipeline, YOLO first localizes the primary product region from the uploaded image. The cropped query image is encoded using the CLIP visual encoder to generate the query embedding. The generated embedding is then compared against indexed catalog embeddings using cosine similarity search to retrieve top matching products. Finally, BLIP-2 image-text matching is used for semantic re-ranking before

displaying the final results to the user.

The overall workflow of the system can be summarized as follows:

1. Product images are collected from the dataset.
2. YOLO detects and crops the clothing region.
3. BLIP-2 generates semantic captions.
4. CLIP generates image and text embeddings.
5. Embeddings are fused and indexed using HNSW.
6. User uploads a query image.
7. Query embedding is generated using CLIP.
8. Similarity search retrieves top matching products.
9. BLIP-2 performs semantic re-ranking.
10. Final results are displayed to the user.

Figure [2.1](#) illustrates the complete architecture of the proposed system.

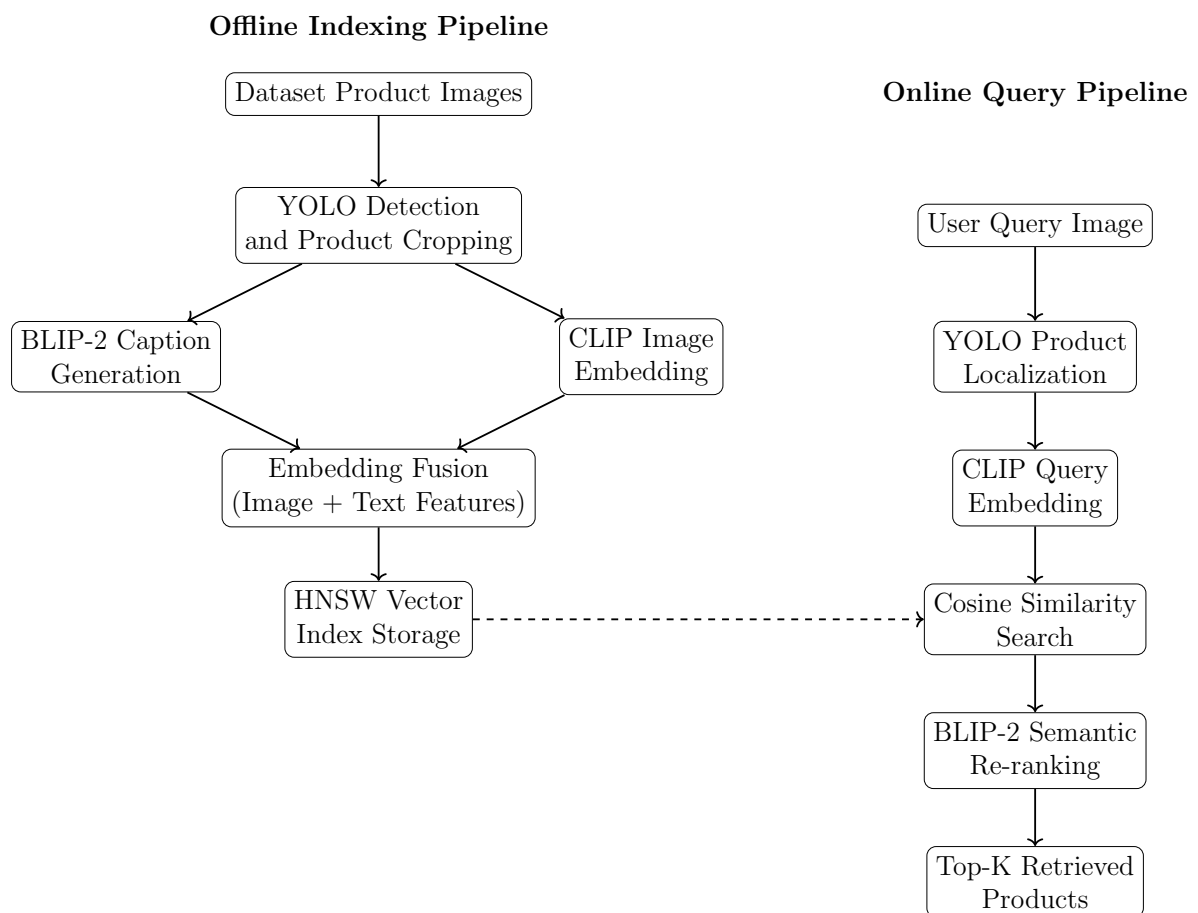


Figure 2.1: Overall Architecture of the Visual Product Search Engine

2.2 Offline Indexing Pipeline

The Offline Indexing Pipeline is responsible for preparing the product database before the retrieval process begins. In this stage, all catalog product images are processed and converted into searchable vector embeddings. The generated embeddings are stored in an HNSW vector index, which enables fast and efficient similarity retrieval during the online search stage.

The main objective of the offline pipeline is to extract meaningful visual and semantic information from each product image and organize it in a format suitable for nearest-neighbor retrieval.

2.2.1 Input to the Pipeline

The offline indexing stage begins with product images collected from the DeepFashion In-Shop Clothes Retrieval dataset. Each image represents a fashion product belonging to a specific item category.

Input:

- Raw catalog product images
- Product metadata and item IDs

Output:

- Preprocessed product images ready for feature extraction

2.2.2 YOLO-based Product Detection

The first major step in the pipeline is product localization using YOLO. The detector identifies the primary clothing object in the image and generates bounding boxes around the product region. The detected region is cropped to remove unnecessary background information such as walls, people, or unrelated objects. This improves the quality of feature extraction and allows the system to focus only on the product.

Purpose of YOLO Detection:

- Reduce background noise
- Improve product visibility

- Increase embedding quality
- Improve retrieval accuracy

Input:

- Raw product image

Output:

- Cropped product image

2.2.3 BLIP-2 Caption Generation

After cropping the product image, the cropped region is passed to the BLIP-2 caption generation model. BLIP-2 generates semantic descriptions for the product based on visual attributes.

The generated captions contain important semantic information such as:

- Product color
- Clothing type
- Style information
- Design attributes

Example captions generated by BLIP-2:

- “Black oversized hoodie”
- “Blue denim jacket with buttons”
- “White printed casual t-shirt”

Input:

- Cropped product image

Output:

- Semantic product caption

2.2.4 CLIP Embedding Generation

CLIP is used to convert both image and text information into vector embeddings in a shared feature space. The cropped image is processed through the CLIP visual encoder, while the generated caption is processed through the CLIP text encoder.

The generated embeddings capture both:

- Visual product appearance
- Semantic product meaning

These embeddings allow cross-modal similarity matching between image and text representations.

Input:

- Cropped product image
- Generated caption

Output:

- Image embedding vector
- Text embedding vector

2.2.5 Embedding Fusion and Normalization

The image embedding and text embedding generated by CLIP are combined into a single fused embedding representation. The fusion step helps the system capture both visual and semantic characteristics of the product.

The fused embedding is then normalized before storage to improve similarity comparison during retrieval.

Purpose of Fusion:

- Combine visual and semantic features
- Improve retrieval relevance
- Increase embedding consistency

Output:

- Final normalized product embedding

2.2.6 HNSW Vector Indexing

The final normalized embeddings are stored in an HNSW (Hierarchical Navigable Small World) vector index. HNSW is an approximate nearest-neighbor search structure designed for efficient large-scale vector retrieval.

The index stores:

- Product embedding vectors
- Product item IDs
- Metadata information

This indexing step allows the retrieval system to quickly search for similar products during the online query stage.

Input:

- Final normalized embeddings

Output:

- Searchable vector database

At the end of the offline indexing pipeline, all catalog products are converted into searchable vector representations and stored inside the HNSW vector database. These indexed embeddings are later used during the online retrieval stage for efficient similarity matching and product recommendation.

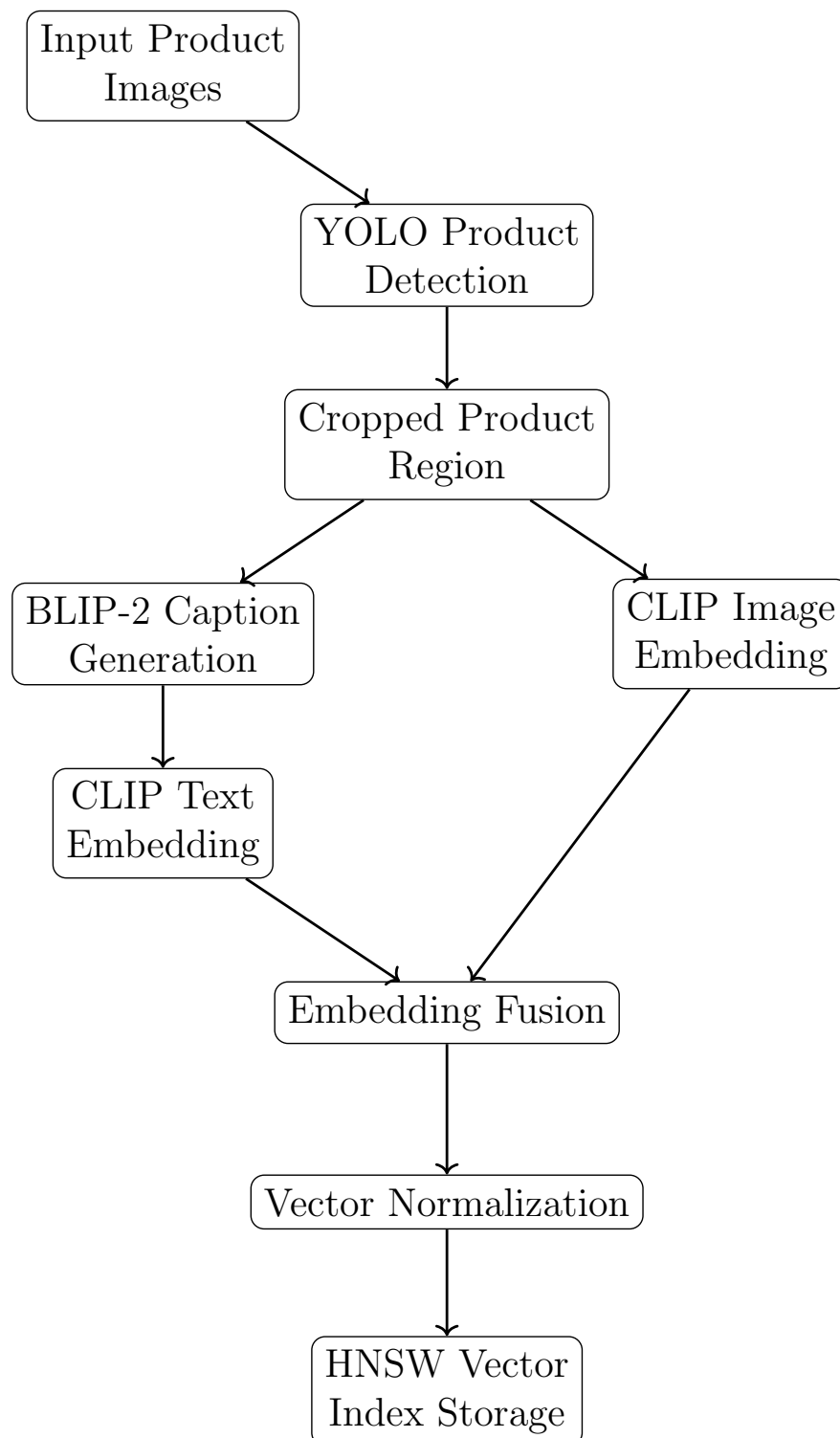


Figure 2.2: Workflow of Offline Indexing Pipeline

2.3 Online Query Pipeline

The Online Query Pipeline is responsible for retrieving visually and semantically similar products for a user-uploaded query image. During this stage, the system processes the query image, generates its embedding representation, searches the indexed vector database, and returns the most relevant matching products.

The main objective of the online retrieval stage is to perform fast and accurate similarity matching between the uploaded query image and the indexed catalog product embeddings generated during the offline indexing stage.

2.3.1 Query Image Upload

The online retrieval process begins when the user uploads a query image through the application interface. The uploaded image may contain clothing products with different backgrounds, lighting conditions, or viewing angles.

Input:

- User-uploaded query image

Output:

- Query image forwarded for preprocessing

2.3.2 Product Localization using YOLO

The uploaded query image is processed using YOLO for product localization. YOLO detects the primary clothing item and generates a bounding box around the product region. The detected region is cropped to remove unnecessary background information and improve feature extraction quality.

Input:

- Raw query image

Output:

- Cropped query product image

2.3.3 CLIP Query Embedding Generation

After preprocessing, the cropped query image is passed through the CLIP visual encoder. CLIP converts the query image into a high-dimensional vector embedding in the shared feature space.

The generated embedding captures important visual features such as:

- Product shape
- Clothing style
- Texture and patterns
- Color information

Input:

- Cropped query image

Output:

- Query embedding vector

2.3.4 Similarity Retrieval

The generated query embedding is compared with the indexed product embeddings stored in the HNSW vector database. Cosine similarity is used to retrieve the most similar products from the catalog. The retrieval system returns top-K candidate products with the highest similarity scores.

Purpose of Similarity Retrieval:

- Fast nearest-neighbor search
- Efficient large-scale retrieval
- Matching visually similar products

Input:

- Query embedding vector
- Indexed catalog embeddings

Output:

- Top-K retrieved candidate products

2.3.5 Semantic Re-ranking

Although similarity retrieval returns visually similar products, some retrieved items may not be semantically accurate. To improve retrieval relevance, BLIP-2 image-text matching is used for semantic re-ranking.

The re-ranking stage evaluates the semantic consistency between the query image and retrieved product captions, and rearranges the retrieved products accordingly.

Purpose of Re-ranking:

- Improve semantic relevance
- Reduce false matches
- Improve final ranking quality

Input:

- Retrieved candidate products
- Product captions

Output:

- Re-ranked product list

2.3.6 Final Retrieval Results

After semantic re-ranking, the final ranked products are displayed to the user through the application interface. The results include the top matching products along with similarity-based ranking.

The online retrieval pipeline enables fast and efficient product search by combining computer vision, semantic understanding, and vector similarity retrieval.

2.3.7 Workflow of Online Query Pipeline

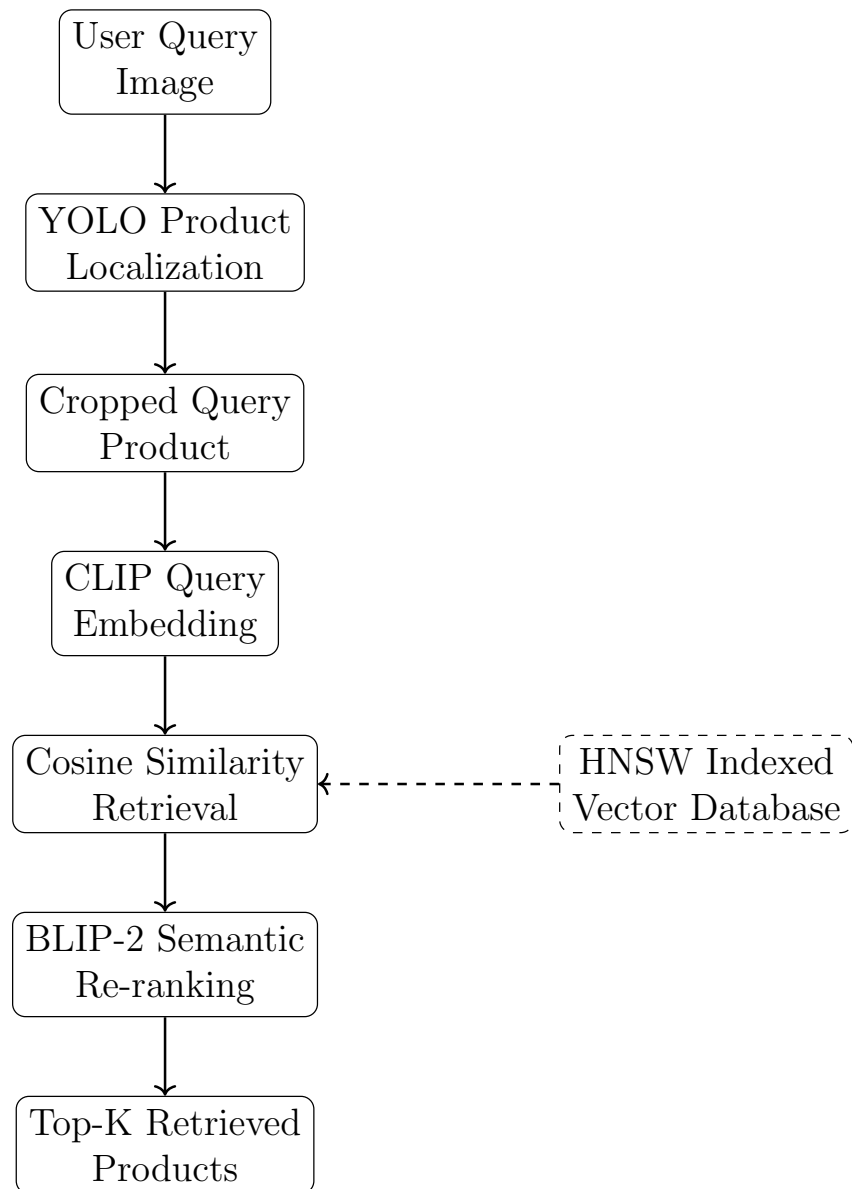


Figure 2.3: Workflow of Online Query Pipeline

Chapter 3

Model and Component Explanation

3.1 YOLO-based Product Detection

YOLO (You Only Look Once) is used in the proposed system for product detection and localization. The primary objective of this stage is to identify the main clothing product present in an image and isolate it from the surrounding background before feature extraction and retrieval.

In fashion product retrieval systems, raw images often contain unnecessary background elements such as walls, furniture, human poses, lighting variations, and unrelated objects. These background components can negatively affect embedding generation and reduce retrieval accuracy. Therefore, product localization is an important preprocessing step in both the offline indexing pipeline and the online query pipeline.

3.1.1 Object Detection using YOLO

YOLO is a real-time object detection model that detects objects directly from an image in a single forward pass. Unlike traditional object detection methods, YOLO performs object localization and classification simultaneously, making it efficient and suitable for large-scale image processing. In this project, YOLO is used to:

- Detect the primary clothing object
- Generate bounding boxes around the product region
- Remove unnecessary background information
- Improve feature extraction quality

The input image is divided into regions, and the model predicts object locations along with confidence scores. The detected region with the highest confidence corresponding to the clothing item is selected for cropping.

3.1.2 Bounding Box Generation

After detecting the clothing product, YOLO generates a bounding box around the object. The bounding box contains the coordinates of the detected product region within the image. The generated bounding box helps:

- Identify the exact product location
- Separate the clothing item from the background
- Focus only on relevant visual information

The cropped region obtained from the bounding box is then forwarded to the subsequent stages such as BLIP-2 caption generation and CLIP embedding extraction.

3.1.3 Importance of Cropping

Cropping plays a significant role in improving the retrieval performance of the system. Instead of processing the entire image, the system processes only the detected product region. Advantages of cropping include:

- Reduction of background noise
- Better visual focus on the product
- Improved embedding consistency
- More accurate similarity retrieval

Without cropping, irrelevant visual information may influence the generated embeddings and produce inaccurate retrieval results.

3.1.4 Background Noise Reduction

Background noise refers to unnecessary image content that does not contribute to the product representation. Examples include:

- Human body posture
- Furniture and room objects
- Lighting variations
- Complex backgrounds

YOLO-based product localization helps reduce this noise by isolating the product region before feature extraction. As a result, the generated embeddings focus mainly on clothing-specific features such as:

- Product shape
- Texture
- Design patterns
- Color information

This improves the overall retrieval quality and semantic consistency of the system.

3.1.5 Before and After Product Cropping

Figure 3.1 shows the comparison between the original product image and the cropped product region generated using YOLO-based detection.

The cropped product image generated by YOLO is later used for semantic caption generation and embedding extraction in the subsequent stages of the retrieval pipeline.

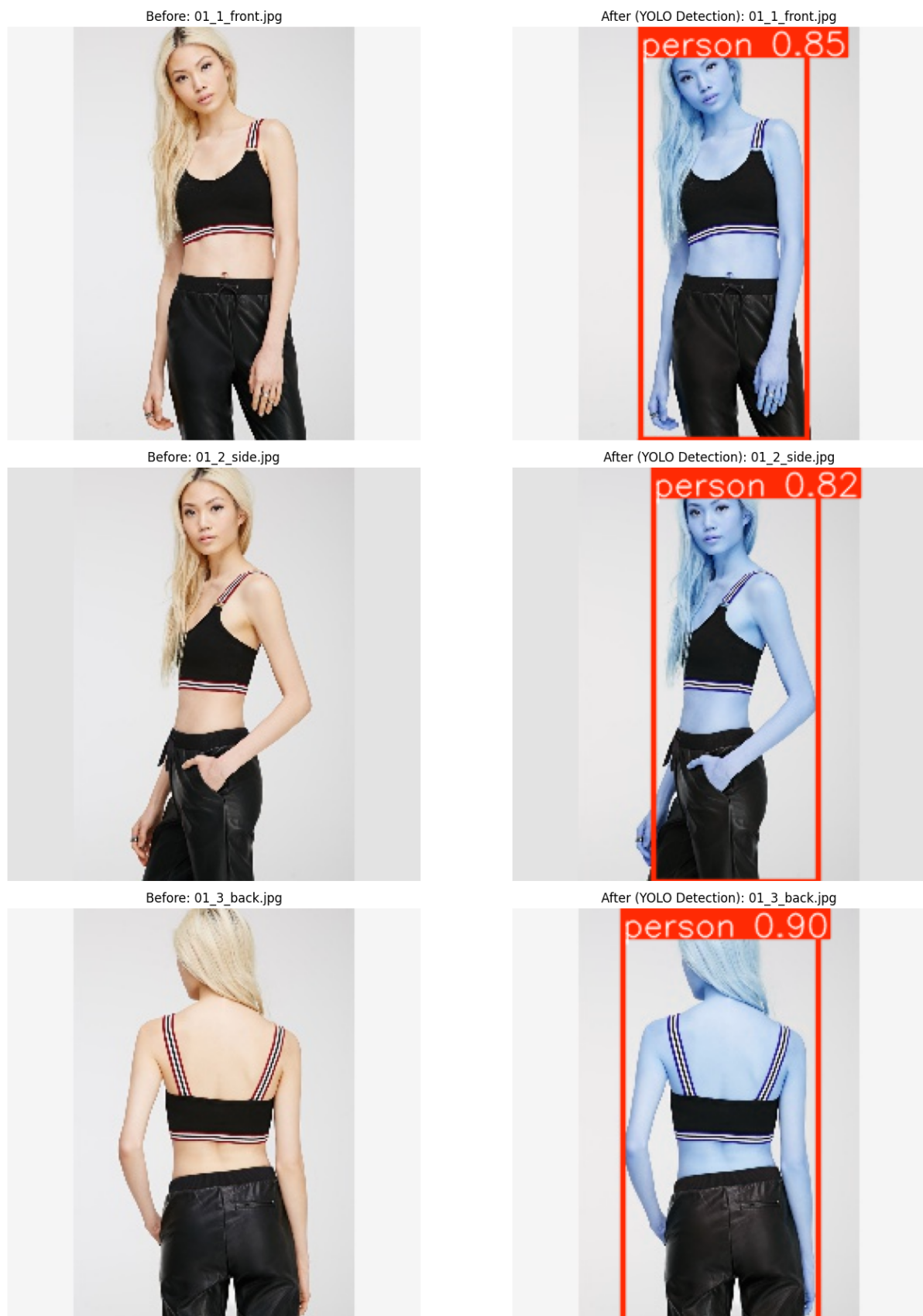


Figure 3.1: Before and After YOLO-based Product Cropping

3.2 BLIP-2 Caption Generation

BLIP-2 (Bootstrapping Language-Image Pre-training) is used in the proposed system for semantic caption generation. The primary objective of this stage is to generate meaningful textual descriptions from product images so that the retrieval system can capture semantic information in addition to visual appearance.

In fashion product retrieval, visual similarity alone may not always provide accurate results. Products with similar shapes may differ in color, style, or clothing type. Therefore, semantic understanding becomes important for improving retrieval relevance. BLIP-2 helps the system understand high-level product attributes by converting image information into descriptive text captions.

3.2.1 Image-to-Text Generation

BLIP-2 is a vision-language model designed for image captioning and multimodal understanding. The model takes an input image and generates a natural language description representing the visual content present in the image. In this project, BLIP-2 receives the cropped product image generated after YOLO-based product detection. The model analyzes the clothing product and produces descriptive captions based on visual features. The generated captions typically include:

- Product type
- Clothing style
- Color information
- Design details
- Texture and appearance

These captions are later converted into text embeddings using the CLIP text encoder.

3.2.2 Semantic Understanding

One of the major advantages of BLIP-2 is its ability to capture semantic meaning from images. Instead of simply identifying objects, the model generates context-aware descriptions that represent the overall appearance and characteristics of the clothing item.

For example:

- “Black oversized hoodie”
- “Blue denim jacket with buttons”
- “White printed casual t-shirt”

These descriptions help the retrieval system understand semantic relationships between products, which improves retrieval accuracy during similarity matching and re-ranking.

3.2.3 Caption Consistency

Traditional e-commerce platforms often suffer from inconsistent product descriptions written by different sellers. Similar products may have different titles and metadata, making keyword-based retrieval unreliable. BLIP-2 helps solve this issue by generating consistent semantic captions directly from visual information. This creates a standardized textual representation for products across the dataset.

Advantages of consistent caption generation include:

- Improved semantic alignment
- Better text embedding quality
- Reduced metadata inconsistency
- Improved retrieval relevance

3.2.4 Workflow of BLIP-2 Caption Generation



The generated semantic captions are later converted into text embeddings using CLIP and combined with image embeddings during the embedding fusion stage.

3.3 CLIP Embedding Model

CLIP (Contrastive Language-Image Pre-training) is the core embedding model used in the proposed Visual Product Search Engine. The primary objective of CLIP is to learn a shared representation space for both images and text so that visually and semantically related products are mapped closer together in the embedding space.

Unlike traditional image retrieval systems that rely only on visual features, CLIP combines image understanding and language understanding together. This enables the retrieval system to capture both visual appearance and semantic meaning during similarity matching.

In this project, CLIP is used for:

- Image embedding generation
- Text embedding generation
- Cross-modal similarity learning
- Semantic similarity retrieval

The model receives:

- Cropped product images from YOLO
- Semantic captions generated by BLIP-2

Both inputs are converted into vector embeddings inside a shared feature space.

3.3.1 Image Encoder

The CLIP image encoder is responsible for extracting visual features from the cropped product image. The encoder analyzes different visual characteristics such as:

- Product shape
- Clothing style
- Texture and patterns
- Color information
- Visual appearance

The image encoder converts the product image into a high-dimensional embedding vector representing the visual content of the product.

In the proposed system, the cropped product image generated after YOLO detection is passed through the CLIP visual encoder to obtain the image embedding.

Input:

- Cropped product image

Output:

- Image embedding vector

3.3.2 Text Encoder

The CLIP text encoder processes the semantic captions generated by BLIP-2. The text encoder converts the generated captions into vector embeddings in the same feature space as the image embeddings.

The text embeddings capture semantic product information such as:

- Clothing type
- Style information
- Product attributes
- Semantic descriptions

For example:

- “Black oversized hoodie”
- “Blue denim jacket with buttons”
- “White printed casual t-shirt”

These semantic descriptions help improve retrieval quality by incorporating language-based understanding into the embedding representation.

Input:

- Generated product caption

Output:

- Text embedding vector

3.3.3 Shared Embedding Space

One of the most important features of CLIP is the shared embedding space. Both image embeddings and text embeddings are projected into the same vector space, allowing the model to directly compare image and text representations.

In this shared feature space:

- Similar products are mapped closer together
- Semantically related image-text pairs have higher similarity
- Unrelated products remain farther apart

This shared embedding representation allows the system to perform cross-modal retrieval, where image information and text information can be compared using similarity metrics.

3.3.4 Cross-modal Learning

CLIP is trained using contrastive learning, where the model learns to align corresponding image-text pairs while separating unrelated pairs.

During training:

- Matching image-text pairs are pulled closer together
- Non-matching pairs are pushed farther apart

This learning strategy helps the model understand semantic relationships between visual content and textual descriptions.

In the proposed system, cross-modal learning improves:

- Semantic understanding
- Retrieval relevance

- Image-text alignment
- Product similarity matching

3.3.5 Similarity Matching Concept

After generating embeddings, similarity comparison is performed using cosine similarity. The query embedding is compared against indexed product embeddings stored in the HNSW vector database.

Products with higher similarity scores are considered more visually and semantically similar to the query image.

The similarity search process enables:

- Fast nearest-neighbor retrieval
- Semantic product matching
- Ranking of similar products

The retrieval system returns the top-K products with the highest similarity scores.

3.3.6 Fine-tuning of CLIP

Pre-trained CLIP models are already capable of learning general image-text relationships. However, fashion product retrieval requires domain-specific understanding of clothing styles, textures, and product categories.

Therefore, CLIP is fine-tuned using the DeepFashion dataset to improve retrieval performance for fashion products.

In the proposed system:

- The CLIP vision encoder is fine-tuned
- BLIP-2 and YOLO remain frozen
- Fine-tuning improves product similarity learning

The fine-tuning process helps:

- Improve embedding quality

- Increase retrieval accuracy
- Improve fashion-specific semantic understanding

3.3.7 Architecture of CLIP Embedding Model

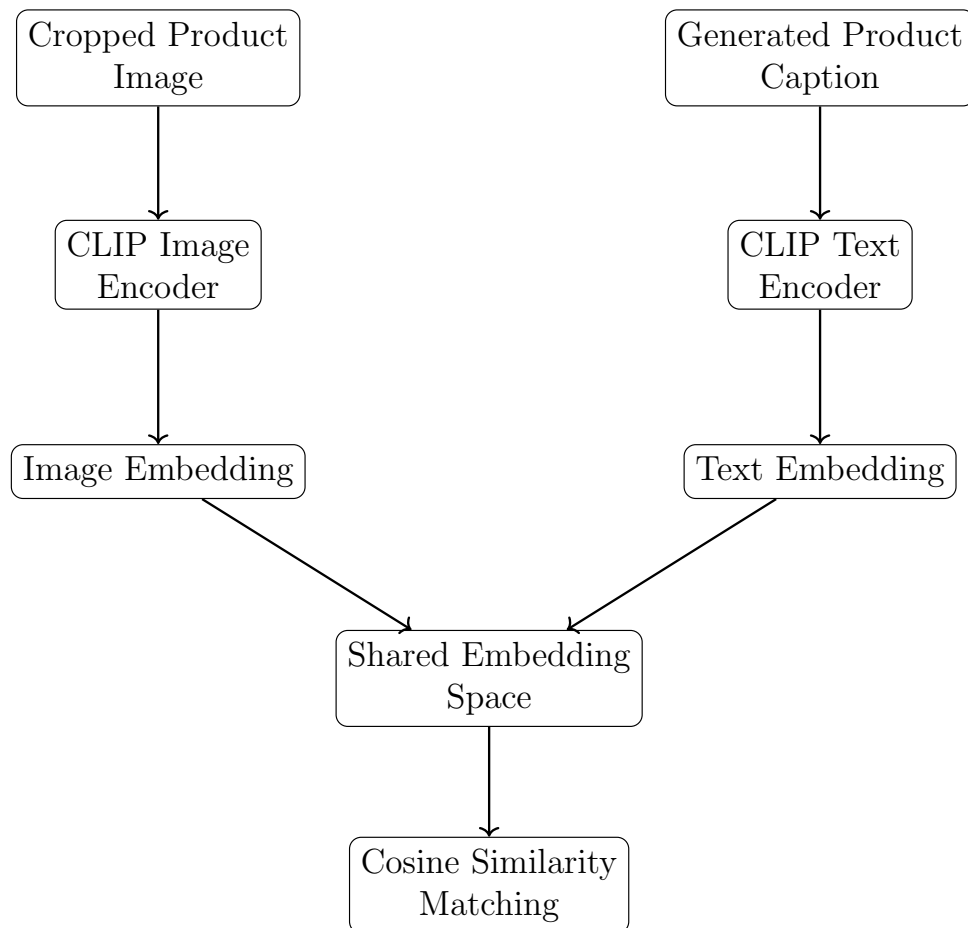
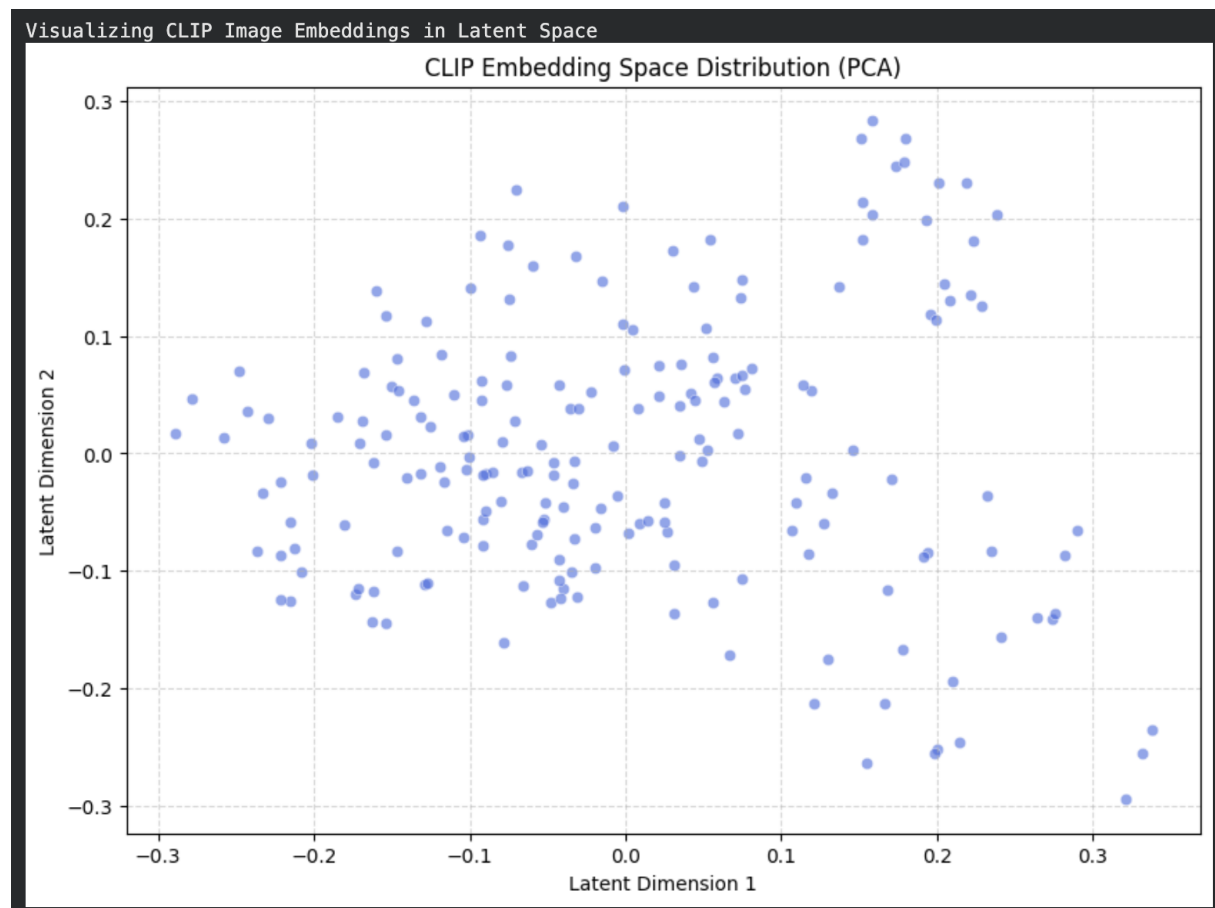


Figure 3.2: Architecture of CLIP Embedding Model



The CLIP embedding model plays a central role in the proposed retrieval system by combining visual understanding and semantic understanding into a unified representation space for efficient product similarity retrieval.

3.4 Embedding Fusion

After generating image embeddings and text embeddings using CLIP, both representations are combined into a single fused embedding vector. The purpose of embedding fusion is to capture both visual appearance and semantic understanding within one unified representation. The image embedding contains visual product information such as shape, texture, color, and design patterns, while the text embedding contains semantic information extracted from the BLIP-2 generated captions. Combining both embeddings improves retrieval quality by incorporating multimodal information into the similarity search process. The fused embedding used in the proposed system is defined as:

$$v_i = \alpha \cdot \phi_V(\hat{x}_i) + (1 - \alpha) \cdot \phi_T(c_i) \quad (3.1)$$

where:

- v_i represents the final fused embedding vector.
- $\phi_V(\hat{x}_i)$ represents the CLIP image embedding.
- $\phi_T(c_i)$ represents the CLIP text embedding.
- α controls the contribution of image and text features.

3.4.1 Image Embedding

The image embedding is generated using the CLIP visual encoder after YOLO-based product cropping. This embedding captures visual characteristics of the clothing product such as:

- Product shape, Texture, Color information, Design patterns, Visual appearance

The visual embedding plays an important role in retrieving visually similar products from the database.

3.4.2 Text Embedding

The text embedding is generated using the CLIP text encoder from captions produced by BLIP-2. These embeddings contain semantic information describing the product.

The text embedding helps the system understand:

- Clothing category
- Style information
- Product attributes
- Semantic descriptions

By including semantic information, the retrieval system becomes more robust to visually similar but semantically different products.

3.4.3 Alpha Weighting

The parameter α determines the relative contribution of image embeddings and text embeddings during fusion.

- Higher α values give more importance to visual features
- Lower α values give more importance to semantic features

Proper selection of α helps balance visual similarity and semantic understanding for improved retrieval performance.

3.4.4 Normalization of Embeddings

After fusion, the final embedding vector is normalized before storage in the HNSW vector database. Normalization ensures that all embeddings have consistent magnitude during similarity comparison.

The normalization process improves:

- Cosine similarity computation
- Embedding consistency
- Retrieval stability
- Nearest-neighbor search performance

The normalized embeddings are then stored in the vector index and later used during online retrieval for similarity matching and ranking.

3.5 HNSW Vector Search

HNSW (Hierarchical Navigable Small World) is used in the proposed system for efficient vector similarity retrieval. After generating the final fused embeddings, all product vectors are stored inside an HNSW-based vector index. During online retrieval, the query embedding is compared against these indexed embeddings to retrieve the most similar products.

Since the product database may contain thousands of embeddings, directly comparing the query vector with every stored embedding becomes computationally expensive. HNSW solves this problem by performing efficient Approximate Nearest Neighbor (ANN) search, which significantly reduces retrieval time while maintaining high retrieval accuracy.

3.5.1 Approximate Nearest Neighbor Search

Nearest Neighbor Search is used to find embeddings that are most similar to the query embedding. Traditional exact nearest-neighbor search compares the query vector with every vector in the database, which becomes slow for large-scale datasets.

HNSW uses Approximate Nearest Neighbor (ANN) search to improve retrieval efficiency. Instead of searching all vectors, the algorithm navigates through graph connections to quickly identify nearby embeddings.

Advantages of ANN search include:

- Faster retrieval speed
- Reduced computational cost
- Scalable large-scale search
- Efficient similarity matching

The retrieval process returns the top-K most similar products based on cosine similarity.

3.5.2 Fast Similarity Retrieval

The primary advantage of HNSW is its ability to perform low-latency vector retrieval even for large embedding databases. During retrieval, the query embedding enters the graph structure and navigates through neighboring nodes to find similar vectors efficiently.

The system retrieves products by:

- Searching nearby embedding regions
- Comparing similarity scores
- Returning top matching products

Fast retrieval is important for real-time product search applications where users expect immediate results after uploading a query image.

3.5.3 Graph-based Indexing Structure

HNSW organizes embeddings using a graph-based indexing structure. Each embedding is represented as a node connected to neighboring embeddings with similar vector representations.

The graph structure contains:

- Multiple hierarchical layers
- Neighbor connections between similar embeddings
- Efficient navigation paths

During search, the algorithm traverses these graph connections to locate embeddings that are closest to the query vector.

The graph-based indexing structure improves:

- Search efficiency
- Scalability
- Retrieval speed
- Similarity matching performance

3.5.4 Workflow of HNSW Vector Search

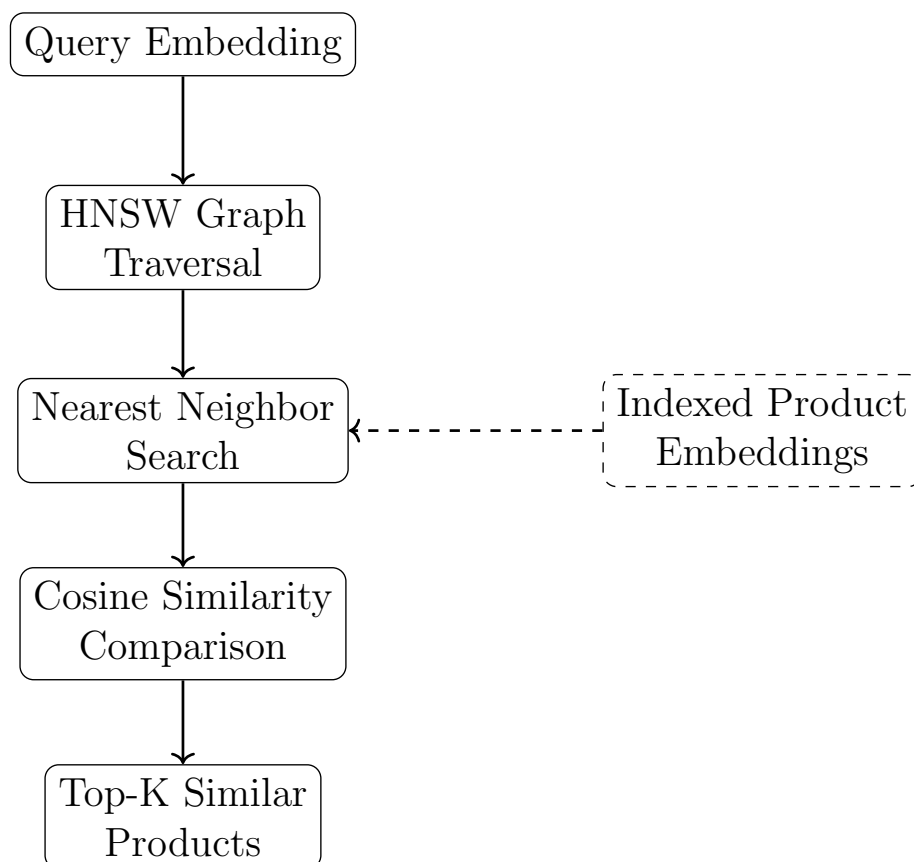


Figure 3.3: Workflow of HNSW Vector Search

```
... --- HNSW Vector Search Database (Metadata) ---
```

	image_path	item_id
0	/content/dataset/train/02_1_front.jpg	id_00000002
1	/content/dataset/train/02_2_side.jpg	id_00000002
2	/content/dataset/train/02_4_full.jpg	id_00000002
3	/content/dataset/train/02_7_additional.jpg	id_00000002
4	/content/dataset/train/02_1_front.jpg	id_00000003

Total vectors in index: 0

3.5.5 HNSW Graph Structure

Figure 3.4 illustrates the graph-based indexing structure used in HNSW. Similar embeddings are connected through graph edges, allowing efficient traversal during nearest-neighbor search.

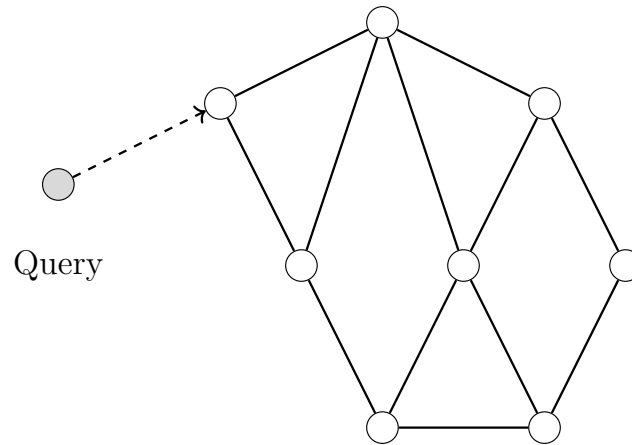


Figure 3.4: Illustration of HNSW Graph-based Indexing

The HNSW vector search mechanism enables efficient large-scale similarity retrieval by organizing embeddings in a graph-based structure and performing fast approximate nearest-neighbor traversal during product search.

3.6 BLIP-2 Semantic Re-ranking

After similarity retrieval using CLIP embeddings and HNSW vector search, the system obtains a list of top-K candidate products. Although these retrieved products are visually similar to the query image, the initial retrieval stage may still contain semantically incorrect or less relevant results.

To improve retrieval quality, the proposed system uses BLIP-2 based semantic re-ranking. This stage evaluates the semantic relationship between the query image and the retrieved product captions and rearranges the candidate products based on semantic relevance.

3.6.1 Limitations of Initial Retrieval

The initial retrieval stage mainly depends on vector similarity between embeddings. While CLIP embeddings capture strong visual and semantic features, some retrieved products may still have:

- Similar colors but different clothing types
- Similar textures but different styles
- Visually related but semantically incorrect products

For example:

- A black hoodie may retrieve a black jacket
- A denim shirt may retrieve denim jeans
- Similar patterns may lead to incorrect product categories

These issues occur because similarity retrieval focuses mainly on embedding proximity in the feature space and may not fully understand detailed semantic relationships.

3.6.2 Semantic Re-ranking Process

To improve the relevance of retrieved products, BLIP-2 semantic re-ranking is applied after the initial similarity retrieval stage.

In this process:

- The query image is compared with retrieved product captions
- BLIP-2 evaluates semantic consistency between image and text
- Candidate products are rearranged based on semantic matching scores

The re-ranking process helps prioritize products that are not only visually similar but also semantically accurate.

Advantages of semantic re-ranking include:

- Improved retrieval relevance
- Better semantic consistency
- Reduction of false matches
- Improved ranking quality

3.6.3 Image-Text Matching (ITM) Score

BLIP-2 uses an Image-Text Matching (ITM) mechanism to evaluate the relationship between the query image and the retrieved product captions.

The ITM score measures:

- Semantic similarity between image and caption
- Contextual consistency
- Relevance of retrieved products

Higher ITM scores indicate stronger semantic alignment between the query image and candidate product descriptions.

The retrieved products are then sorted according to their ITM scores, producing a more accurate final ranking.

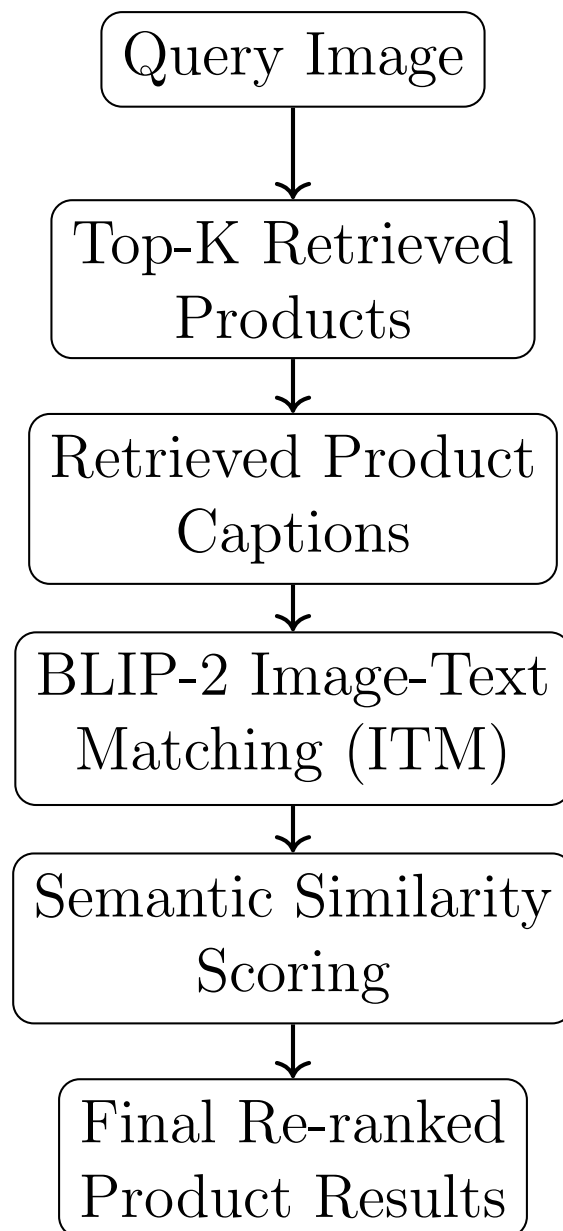
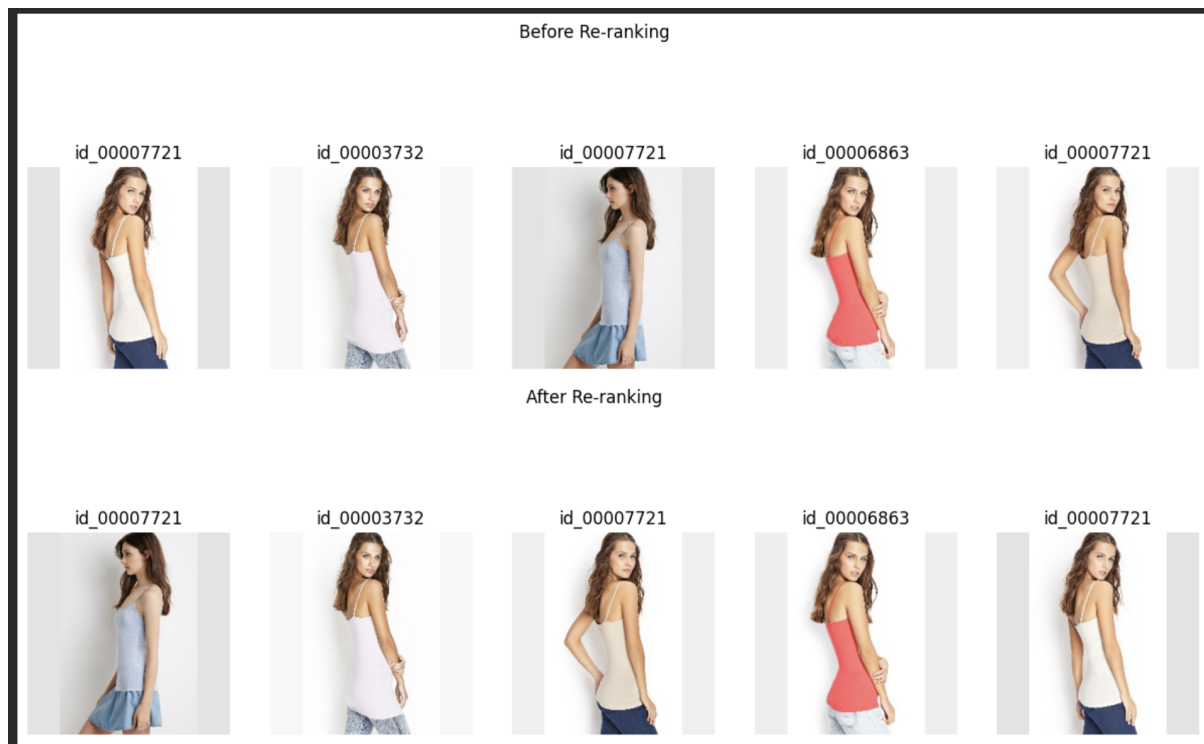


Figure 3.5: Workflow of BLIP-2 Semantic Re-ranking

3.6.4 Workflow of BLIP-2 Re-ranking



The semantic re-ranking stage significantly improves the overall retrieval performance of the proposed system by combining visual similarity with semantic understanding before generating the final ranked search results.

Chapter 4

Results and Discussion

This chapter presents the retrieval performance of the proposed Visual Product Search Engine on the DeepFashion dataset. The system was evaluated using Recall@K, mAP@K, and NDCG@K metrics.

4.1 Retrieval Performance

K	Recall@K	mAP@K	NDCG@K
5	0.6215	0.4125	0.4751
10	0.7396	0.4020	0.5070
15	0.7847	0.3846	0.5149

Table 4.1: Retrieval Performance Results

The results show that Recall@K improves as the value of K increases, indicating that the system retrieves more relevant products within larger retrieval ranges. NDCG@K also increases gradually, showing improved ranking quality of relevant products.

4.2 Top-K Retrieval Examples

Figure 4.1 shows example retrieval outputs generated by the proposed system.

4.3 Good Retrieval Cases

The system performs well when:

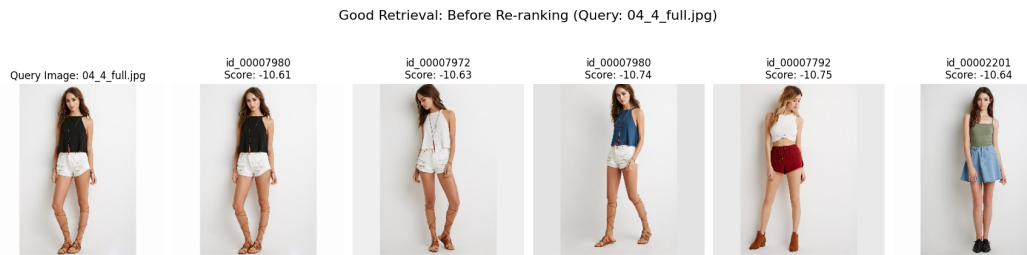


Figure 4.1: Top-K Retrieval Results

- Product images are clear
- Background noise is minimal
- Clothing textures and colors are distinctive

The combination of CLIP embeddings and BLIP-2 semantic understanding improves retrieval relevance.

4.4 Failure Cases

Some incorrect retrievals were observed when products had:

- Similar colors
- Similar textures
- Complex backgrounds

In some cases, visually similar products from different clothing categories were retrieved.

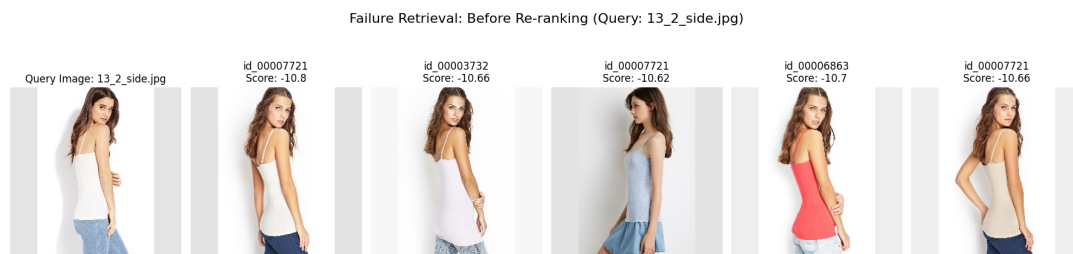


Figure 4.2: Examples of Retrieval Failure Cases

Chapter 5

UI and Deployment

5.1 Objective

The objective of this phase was to develop an interactive user interface for the visual product retrieval system and deploy it as a publicly accessible web application. The deployment enables users to upload a query image, automatically detect clothing regions, select the desired clothing item, and retrieve visually similar products from the gallery dataset.

The deployed application integrates the following major components:

- YOLOv8 for person detection and clothing region extraction.
- Fine-tuned CLIP model for feature extraction.
- FAISS index for efficient similarity search.
- Streamlit for the frontend user interface.
- Hugging Face Dataset Repository for hosting model assets and gallery data.
- Streamlit Community Cloud for public deployment.

5.2 System Architecture

The complete deployment pipeline is illustrated below:

**User Image Upload → YOLO Detection → Clothing Region Selection →
CLIP Embedding Generation → FAISS Retrieval → Display Retrieved
Products**

The application first accepts an uploaded image from the user. YOLOv8 detects the person present in the image, after which the system automatically generates different clothing regions such as:

- Full Body
- Upper Body
- Lower Body

The user can select any of these regions for retrieval. The selected region is then passed through the fine-tuned CLIP model to generate feature embeddings. These embeddings are searched against the FAISS index to retrieve visually similar products from the gallery dataset.

5.3 Frontend Development Using Streamlit

The user interface was implemented using **Streamlit**. Streamlit was chosen because of its lightweight architecture, ease of deployment, and ability to rapidly build machine learning interfaces.

The Streamlit application performs the following tasks:

1. Accepts user image upload.
2. Displays the uploaded image.
3. Runs YOLOv8 inference for person detection.
4. Automatically crops multiple clothing regions.
5. Displays region selection buttons.
6. Generates CLIP embeddings for the selected region.
7. Performs similarity retrieval using FAISS.
8. Displays top retrieved products along with similarity scores.

The interface was designed to satisfy the project requirement of allowing the user to explicitly choose which clothing item to search from the uploaded image.

5.3.1 Automatic Clothing Region Extraction

A pre-trained **YOLOv8** model was used for person detection during inference. Instead of manually cropping clothing items, the application performs automatic region extraction.

The following steps are executed:

1. YOLOv8 detects the person bounding box.
2. The detected person region is cropped.
3. The cropped region is divided into:
 - Upper body region
 - Lower body region
 - Full body region
4. These regions are displayed to the user.
5. The selected region is used for retrieval.

This approach enables the system to retrieve specific clothing items such as jackets, shirts, jeans, or dresses from full-body images.

5.3.2 Feature Extraction and Retrieval

The retrieval pipeline uses a fine-tuned CLIP model trained during earlier phases of the project.

The workflow is as follows:

1. The selected clothing crop is processed using the CLIP processor.
2. The CLIP vision encoder generates image embeddings.
3. Embeddings are normalized.
4. The normalized embedding is searched against a precomputed FAISS index.
5. Top-K similar products are retrieved.
6. Retrieved gallery images are displayed in the Streamlit interface.

The FAISS index enables efficient nearest-neighbor retrieval even for large embedding datasets.

5.3.3 Hugging Face Dataset Hosting

Initially, Google Drive was used for storing model assets and gallery data. However, this approach caused deployment issues due to:

- ZIP extraction failures
- Download token limitations
- Large file handling issues
- Streamlit deployment instability

To solve these issues, the assets were migrated to a **Hugging Face Dataset Repository**. This provided reliable hosting for large machine learning assets.

The following files and folders were uploaded to the Hugging Face dataset repository:

- `clip_finetuned/`
 - Fine-tuned CLIP model weights
 - Model configuration files
- `embeddings/`
 - FAISS index files
 - Precomputed image embeddings
 - Metadata files
- `gallery.zip`
 - Gallery product images used for retrieval

The Hugging Face dataset repository was then accessed directly from the Streamlit application using the `huggingface_hub` library.

5.3.4 Asset Download and Caching

During application startup, the following operations are performed:

1. Download CLIP model and embedding assets using `snapshot_download()`.
2. Download `gallery.zip` using `hf_hub_download()`.

3. Extract gallery images locally.
4. Load the FAISS index and metadata.
5. Build image path mappings dynamically.

To reduce repeated downloads and improve performance, Streamlit caching was used through:

- `@st.cache_resource`

This significantly reduced loading time after the first application startup.

5.4 Deployment on Streamlit Community Cloud

The application was deployed publicly using **Streamlit Community Cloud**. The deployment process involved:

1. Creating a GitHub repository.
2. Uploading:
 - `app.py`
 - `requirements.txt`
 - `packages.txt`
3. Connecting the GitHub repository to Streamlit Cloud.
4. Installing required Python dependencies.
5. Installing required Linux system libraries.
6. Launching the application publicly.

The `requirements.txt` file contained Python dependencies such as:

- Streamlit
- PyTorch
- Transformers
- FAISS

- Ultralytics YOLO
- Hugging Face Hub

The `packages.txt` file was used to install Linux-level dependencies required by OpenCV and YOLO.

5.4.1 Final Output

The final deployed system provides:

- Interactive image upload interface
- Automatic clothing region detection
- User-selectable clothing retrieval
- Fast similarity search using FAISS
- Publicly accessible deployment
- End-to-end visual product search pipeline

The deployed application successfully satisfies the project requirements for automatic clothing item retrieval from unseen full-body images.

5.5 User Interface Demonstration

This section presents screenshots of the deployed Streamlit application demonstrating the complete retrieval workflow.

User Uploading Query Image

Uploaded Image ⇌



Figure 5.1: User uploading a query image into the Streamlit application

Automatic Clothing Region Extraction

After image upload, the system automatically detects the person and generates multiple clothing regions including full body, upper body, and lower body. The user can then select the desired region for retrieval.

Choose Clothing Region ⇌



Search Full Body



Search Upper Body



Search Lower Body

Figure 5.2: Automatically generated Full Body, Upper Body, and Lower Body regions

Retrieved Similar Products

Once the user selects a clothing region, the system generates CLIP embeddings and performs similarity retrieval using FAISS. The top retrieved visually similar products are displayed along with similarity scores.

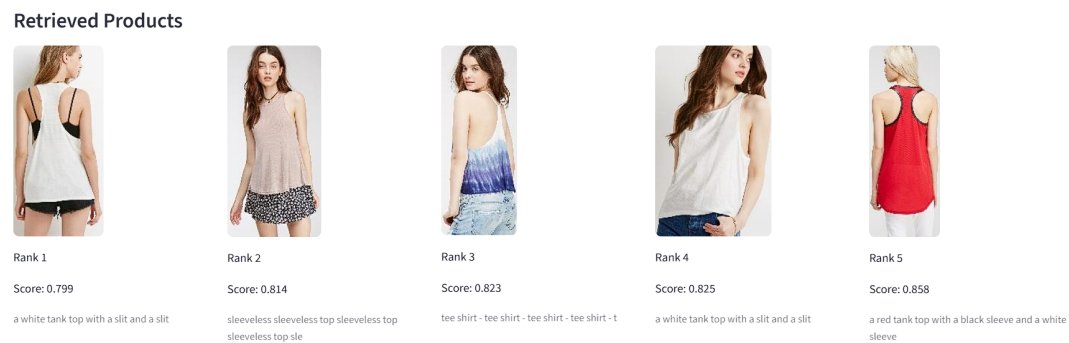


Figure 5.3: Top-5 retrieved similar products displayed by the system

Handling Invalid Inputs

The system also handles cases where no person is detected in the uploaded image. Since the retrieval pipeline is designed for clothing-based retrieval, the application first performs person detection using YOLOv8. If no person is detected, the system displays an appropriate error message to the user instead of proceeding with retrieval. This prevents invalid feature extraction and improves robustness of the application.

Visual Product Search Engine

Upload a full-body image and choose which clothing region to search.

[Upload Full Image](#)

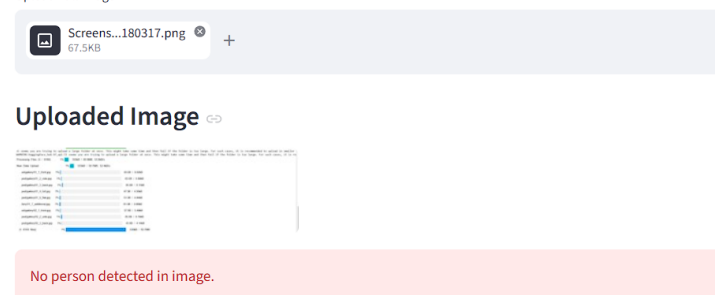


Figure 5.4: Error message

5.6 Project Links

GitHub Repository:

<https://github.com/PrashantSharma30/VR-Project>

Streamlit Deployment Link:

<https://vr-project-ixunkpu7g2estxzalnef2n.streamlit.app/>

Hugging Face Dataset Repository:

<https://huggingface.co/datasets/Sharma-30/fashion-retrieval-assets>

Google Colab Notebook Link:

https://github.com/PrashantSharma30/VR-Project/blob/main/Copy_of_VR_Final_Project.ipynb